

Practical: 01

Aim: Write a program to implement Tic-Tac-Toe game problem.

Code:

```
#include<stdio.h>

char board[9]={'1','2','3','4','5','6','7','8','9'};

void displayBoard(){

    printf("\n");

    printf("%c |%c |%c\n",board[0],board[1],board[2]);

    printf("--|--\n");

    printf("%c |%c |%c\n",board[3],board[4],board[5]);

    printf("--|--\n");

    printf("%c |%c |%c \n",board[6],board[7],board[8]);

}

int checkWin(char player){

    int i;

    int win[8][9]={ {0,1,2},{3,4,5},{6,7,8},

                    {0,3,6},{1,4,7},{2,5,8},

                    {0,4,8},{2,4,6} };

    for(i=0;i<8;i++){

        if(board[win[i][0]]==player&&

           board[win[i][1]]==player&&

           board[win[i][2]]==player)

            return 1;

    }

}
```

```
    return 0;
}

int checkDraw(){
    int i;
    for(i=0;i<9;i++){
        if(board[i]!='X'&&board[i]!='O')
            return 0;
    }
    return 1;
}

void computerMove(){
    int i;
    for(i=0;i<9;i++){
        if(board[i]!='X'&& board[i]!='O'){
            char temp=board[i];
            board[i]='O';
            if(checkWin('O'))
                return;
            board[i]=temp;
        }
    }
    for(i=0;i<9;i++){
        if(board[i]!='X'&& board[i]!='O'){
            char temp=board[i];
```

```
        board[i]='X';

        if(checkWin('X')){

            board[i]='O';

            return;

        }

        board[i]=temp;

    }

}

for(i=0;i<9;i++){

    if(board[i]!='X'&&board[i]!='O'){

        board[i]='O';

        return;

    }

}

}

int main(){

    int choice;

    printf("SINGLE PLAYER TIC TAC TOE \n");

    printf("You are X|Computer is O\n");

    while(1){

        displayBoard();

        printf("Enter your move(1-9):");

        scanf("%d",&choice);

        if(choice<1 || choice>9 ||board[choice-1]=='X'||board[choice-1]=='O'){
```

```
        printf("Invalid move try again\n");
        continue;
    }
    board[choice-1]='X';
    if(checkWin('X')){
        displayBoard();
        printf("You Win!!");
        break;
    }
    if(checkDraw()){
        displayBoard();
        printf("It's a Draw!\n");
        break;
    }
    computerMove();
    if(checkWin('O')){
        displayBoard();
        printf("Computer Win!\n");
        break;
    }
    if(checkDraw()){
        displayBoard();
        printf("It's a Draw!\n");
        break;
    }
```

```
    }  
}  
  
return 0;  
  
}
```

Output:

```
O |X |3  
--|--|--  
4 |5 |6  
--|--|--  
7 |8 |9  
Enter your move(1-9):6  
  
O |X |O  
--|--|--  
4 |5 |X  
--|--|--  
7 |8 |9  
Enter your move(1-9):2  
Invalid move try again  
  
O |X |O  
--|--|--  
4 |5 |X  
--|--|--  
7 |8 |9  
Enter your move(1-9):5  
  
O |X |O  
--|--|--  
O |X |X  
--|--|--  
7 |8 |9  
Enter your move(1-9):8  
  
O |X |O  
--|--|--  
O |X |X  
--|--|--  
7 |X |9  
You Win!!
```

Practical: 02

Aim: Write a program to implement BFS (for 8 puzzle problem or Water Jug problem or any AI search problem).

Code:

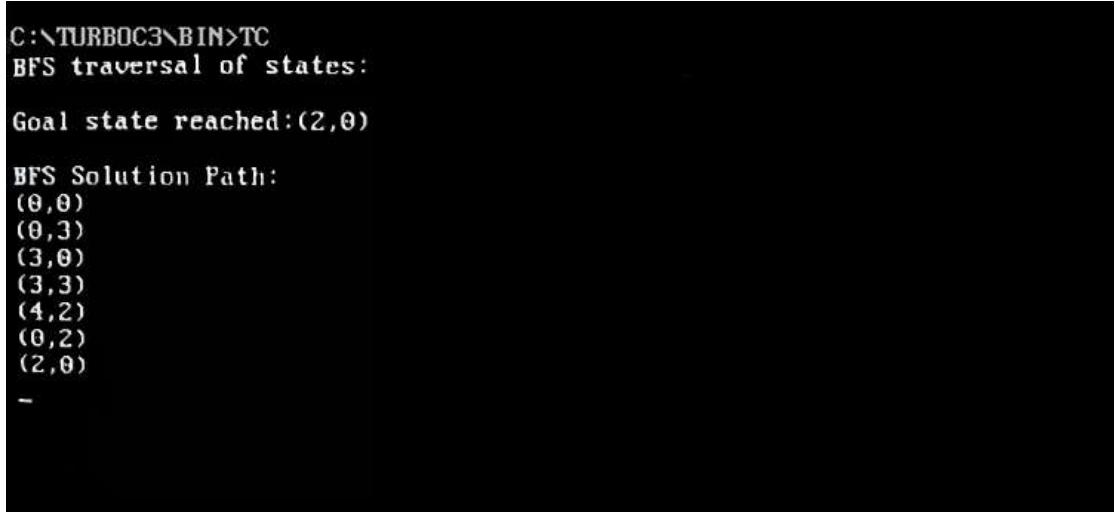
```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#define MAX 100
typedef struct{
    int x;
    int y;
}State;
State queue[MAX];
int front=-1;
int rear=-1;
int visited[5][4];
void enqueue(State s){
    if(rear==MAX-1)
        return;
    if(front==-1)
        front=0;
    queue[++rear]=s;
}
State dequeue(){
    return queue[front++];
}
int isEmpty(){
    return front>rear;
}
```

```

void BFS(){
    State start={0,0};
    enqueue(start);
    visited[0][0]=1;
    printf("BFS traversal of states:\n");
    while(!isEmpty()){
        State curr=dequeue();
        int x=curr.x;
        int y=curr.y;
        printf("(%d,%d)\n",x,y);
        if(x==2){
            printf("\nGoal state reached:(%d,%d)\n",x,y);
            return;
        }
        State nextStates[6] = {
            {4,y},
            {x,3},
            {0,y},
            {x,0},
            {x - (3-y < x ? 3-y:x), y + (3-y < x ? 3-y:x)},
            {x + (4-x < y ? 4-x:y), y - (4-x < y ? 4-x:y)}
        };
        for(int i=0;i<6;i++){
            int nx=nextStates[i].x;
            int ny=nextStates[i].y;
            if(!visited[nx][ny]){
                visited[nx][ny]=1;
                enqueue(nextStates[i]);
            }
        }
    }
}

```

```
    }  
    printf("\nGoal state not found\n");  
}  
int main(){  
    BFS();  
    return 0;  
}
```

Output:

```
C:\TURBOC3\BIN>TC  
BFS traversal of states:  
  
Goal state reached:(2,0)  
  
BFS Solution Path:  
(0,0)  
(0,3)  
(3,0)  
(3,3)  
(4,2)  
(0,2)  
(2,0)  
-
```

Practical: 03

Aim: Write a program to implement DFS (for 8 puzzle problem or Water Jug problem or any AI search problem).

Code:

```
#include<stdio.h>

#include<conio.h>

#define MAX 100

typedef struct{

    int x;

    int y;

}State;

State stack[MAX];

int top=-1;

int visited[5][4];

void push(State s){

    if(top<MAX-1)

        stack[++top]=s;

}

State pop(){

    return stack[top--];

}

int isEmpty(){

    return top==-1;

}void DFS(){
```

```
State start={0,0};

int x,y,i,nx,ny;

State nextStates[6];

State curr;

push(start);

printf("DFS traversal of states:\n");

while(!isEmpty()){

    curr=pop();

    x=curr.x;

    y=curr.y;

    if(visited[x][y])

        continue;

    visited[x][y]=1;

    printf("(%d,%d)\n",x,y);

    if(x==2){

        printf("\nGoal state reached:(%d,%d)\n",x,y);

        return;

    }

    nextStates[0].x=4;

    nextStates[0].y=y;

    nextStates[1].x=x;

    nextStates[1].y=3;

    nextStates[2].x=0;

    nextStates[2].y=y;
```

```

    nextStates[3].x=x;

    nextStates[3].y=0;

    nextStates[4].x=x-(x<(3-y)?x:(3-y));
    nextStates[4].y=y+(x<(3-y)?x:(3-y));

    nextStates[5].x=x+(y<(4-x)?y:(4-x));
    nextStates[5].y=y-(y<(4-x)?y:(4-x));

    for(i=0;i<6;i++){

        nx=nextStates[i].x;
        ny=nextStates[i].y;

        if(!visited[nx][ny]){

            push(nextStates[i]);

        }

    }

    printf("\nGoal state not found\n");

}

int main(){

    DFS();

    return 0;

}

```

Output:

```

C:\TURBOC3\BIN>TC
Goal state reached:(2,0)

DFS Solution Path:
(0,0)
(0,3)
(3,0)
(3,3)
(4,2)
(0,2)
(2,0)
_

```

Practical: 04

Aim: Write a program to implement Single Player Game (Using heuristic function).

Code:

```
#include <stdio.h>

#include<stdlib.h>

#include<conio.h>

#define MAX 100

#define VISITED_MAX 100

int visited[VISITED_MAX][3][3];

int visitedCount = 0;

int goal[3][3] = {{ '1', '2', '3'},
                  { '4', '5', '6'},
                  { '7', '8', '0'} };

typedef struct {

    int state[3][3];

    int h;

} Node;

Node pq[MAX];

int size = 0;

int calculateHeuristic(int state[3][3]) {

    int count = 0;

    int i,j;

    for (i = 0; i < 3; i++) {
```

```
    for (j = 0; j < 3; j++) {  
        if (state[i][j] != 0 && state[i][j] != goal[i][j]) {  
            count++;  
        }  
    }  
}  
  
return count;  
}  
  
void insert(Node n) {  
    pq[size++] = n;  
}  
  
Node removeBest(){  
    int best = 0;  
    int i;  
    Node bestNode;  
    for (i = 1; i < size; i++) {  
        if (pq[i].h < pq[best].h) {  
            best = i;  
        }  
    }  
    bestNode = pq[best];  
    for (i = best; i < size - 1; i++) {  
        pq[i] = pq[i + 1];  
    }  
    size--;  
}
```

```
    return bestNode;
}int isGoal(int state[3][3]) {
    int i,j;
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            if (state[i][j] != goal[i][j]) {
                return 0;
            }
        }
    }
    return 1;
}
void printState(Node n) {
    int i,j;
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            printf("%d ", n.state[i][j]);
        }
        printf("\n");
    }
    printf("h=%d", n.h);
    printf("\n\n");
}
void copyState(int src[3][3], int dest[3][3]) {
```

```
int i,j;

for (i = 0; i < 3; i++) {

    for (j = 0; j < 3; j++) {

        dest[i][j] = src[i][j];

    }

}

}

int sameState(int a[3][3], int b[3][3]) {

    int i,j;

    for (i = 0; i < 3; i++) {

        for (j = 0; j < 3; j++) {

            if (a[i][j] != b[i][j]) {

                return 0;

            }

        }

    }

    return 1;

}

int isVisited(int state[3][3]) {

    int i;

    for (i = 0; i < visitedCount; i++) {

        if (sameState(state, visited[i])) {

            return 1;

        }

    }

}
```

```
    }

    return 0;
}

void markVisited(int state[3][3]) {
    int i,j;

    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            visited[visitedCount][i][j] = state[i][j];
        }
    } visitedCount++;
}

void bestFirstSearch(int start[3][3]) {
    Node startNode;

    int x,y,i;

    int dx[] = {-1, 1, 0, 0};
    int dy[] = {0, 0, -1, 1};

    copyState(start, startNode.state);

    startNode.h = calculateHeuristic(start);

    insert(startNode);

    printf("\nSolution path (State Expanded): \n\n");

    while (size > 0) {
        Node current = removeBest();

        printState(current);

        if(isVisited(current.state)) {
```

```
        continue;
    }

    markVisited(current.state);

    if (isGoal(current.state)) {

        printf("Goal state reached!\n");

        return;
    } for(x=0;x<3;x++){

        for(y=0;y<3;y++){

            if(current.state[x][y] == 0)

                goto found;

        }

    }

found:

for (i = 0; i < 4; i++) {

    int nX = x + dx[i];

    int nY = y + dy[i];

    if (nX >= 0 && nX < 3 && nY >= 0 && nY < 3) {

        Node child;

        copyState(current.state, child.state);

        child.state[x][y] = child.state[nX][nY];

        child.state[nX][nY] = 0;

        child.h = calculateHeuristic(child.state);

        if (!isVisited(child.state)) { insert(child);

        }

    }

}
```

```
    }  
    }  
}  
  
printf("Goal state not reached.\n");  
  
}  
  
int main() {  
  
    int start[3][3];  
  
    int i,j;  
  
    printf("Enter the initial state (0 for blank):\n");  
  
    for(i=0;i<3;i++){  
  
        for(j=0;j<3;j++){  
  
            scanf("%d",&start[i][j]);  
  
        }  
  
    }  
  
    bestFirstSearch(start);  
  
}
```

Output:

```
Enter Initial State(0 for blank):  
1 2 3  
4 5 6  
7 0 8  
  
Solution Path(State Expanded):  
1 2 3  
4 5 6  
7 0 8  
h=1  
1 2 3  
4 5 6  
7 8 0  
h=0  
  
Goal state is reached.
```

Practical 5

Aim: Write a program to Implement A* Algorithm.

```
#include<stdio.h>

#include<stdlib.h>

#include<conio.h>

#define MAX 100

#define VISITED_MAX 100

int visited[VISITED_MAX][3][3];

int visitedCount = 0;

int goal[3][3]={

    {1,2,3},

    {4,5,6},

    {7,8,0}};

typedef struct{

    int state[3][3];

    int h,g,f;

}Node;

Node pq[MAX];

int size = 0;

int calculateHeuristic(int state[3][3]){

    int count = 0;

    int i,j;

    for(i=0;i<3;i++){

        for(j=0;j<3;j++){
```

```
        if(state[i][j] !=0 && state[i][j] != goal[i][j]){  
            count++;  
        }  
    }  
}  
  
return count;  
}  
  
void insert(Node n){  
    pq[size++]=n;  
}  
  
Node removeBest(){  
    int best=0;  
    int i;  
    Node bestNode;  
    for(i=1;i<size;i++){  
        if(pq[i].f<pq[best].f){  
            best = i;  
        }  
    }  
    bestNode = pq[best];  
    for(i=best;i<size-1;i++){  
        pq[i] = pq[i+1];  
    }size--;  
    return bestNode;  
}  
  
int isGoal(int state[3][3]){
```

```
int i,j;

for(i=0;i<3;i++){

    for(j=0;j<3;j++){

        if(state[i][j] != goal[i][j]){

            return 0;

        } }

    return 1;

}

void printState(Node n){

int i,j;

for(i=0;i<3;i++){

    for(j=0;j<3;j++){

        printf("%d ",n.state[i][j]);

    }

    printf("\n");

}

printf("h= %d ",n.h);

printf("g= %d ",n.g);

printf("f= %d ",n.f);

printf("\n\n");

}

void copyState(int src[3][3], int dest[3][3]){

    int i,j;

    for(i=0;i<3;i++){
```

```
        for(j=0;j<3;j++){
            dest[i][j] = src[i][j];
        }
    }

int sameState(int a[3][3],int b[3][3]){
    int i,j;

    for(i=0;i<3;i++){
        for(j=0;j<3;j++){
            if(a[i][j] != b[i][j])
                return 0;
        }
    }

    return 1;
}

int isVisited(int State[3][3]){
    int i;

    for(i=0;i<visitedCount;i++){
        if(sameState(State,visited[i]))
            return 1;
    }

    return 0;
}

void markVisited(int state[3][3]){
    int i,j;
```

```
for(i=0;i<3;i++){  
    for(j=0;j<3;j++){  
        visited[visitedCount][i][j]=state[i][j];  
    }  
}  
  
visitedCount++;  
  
}  
  
void Astar(int start[3][3]){  
    int x,y,i;  
  
    Node startNode;  
  
    int dx[]={-1,1,0,0};  
    int dy[]={0,0,-1,1};  
  
    copyState(start,startNode.state);  
  
    startNode.h=calculateHeuristic(start);  
  
    startNode.g=0;  
  
    startNode.f=startNode.g+startNode.h;  
  
    insert(startNode);  
  
    printf("\nSolution Path(State Expanded):\n\n");  
  
    while(size>0){  
        Node current = removeBest();  
  
        printState(current);  
  
        if(isVisited(current.state))  
  
            continue;  
  
        markVisited(current.state);  
    }  
}
```

```
    if(isGoal(current.state)){  
  
        printf("Goal state is reached.\n");  
  
        return;  
  
    }  
  
    for(x=0;x<3;x++){  
for(y=0;y<3;y++){  
    if(current.state[x][y]==0)  
        goto found;  
    }  
}  
  
    found:  
    for(i=0;i<4;i++){  
int nx= x + dx[i];  
int ny= y + dy[i];  
  
    if(nx>=0 && nx<3 && ny>=0 && ny<3){  
        Node child;  
  
        copyState(current.state,child.state);  
  
        child.state[x][y]=child.state[nx][ny];  
  
        child.state[nx][ny]=0;  
  
        child.g=current.g+1;  
  
        child.h=calculateHeuristic(child.state);  
  
        child.f=child.g+child.h;  
  
        if(!isVisited(child.state)){  
            insert(child);}
```

```

    } }

}

printf("Goal State is Not Reached.\n");

}void main(){

int start[3][3];

int i,j;

clrscr();

printf("Enter Initial State(0 for blank):\n");

for(i=0;i<3;i++){

    for(j=0;j<3;j++){

        scanf("%d",&start[i][j]);

    }

    printf("\n");

}

Astar(start);

getch();

```

OUTPUT:

```

Enter Initial State(0 for blank):
1 2 3
4 5 6
7 0 8

Solution Path(State Expanded):
1 2 3
4 5 6
7 0 8
h= 1 g= 0 f= 1
1 2 3
4 5 6
7 8 0
h= 0 g= 1 f= 1
Goal state is reached.

```

Practical 06

Aim: Write a program to solve N-Queens problem using Prolog.

% N-Queens Problem

% Entry point

nqueens(N, Solution) :-

length(Solution, N),

place_queens(Solution, N).

% Place queens recursively

place_queens([], _).

place_queens([Q|Others], N) :-

place_queens(Others, N),

between(1, N, Q),

safe(Q, Others, 1).

% Check if a queen is safe

safe(_, [], _).

safe(Q, [Q1|Others], D) :-

Q $\neq$ Q1, % Not in same column

$\text{abs}(Q - Q1) \neq D$, % Not in same diagonal

D1 is D + 1,

safe(Q, Others, D1).

```
| ?- nqueens(4, S).
S = [3, 1, 4, 2] ? ;
S = [2, 4, 1, 3] ? ;
```

Practical 07

Aim: Write a program to solve 8 puzzle problem using Prolog.

% Goal State

```
goal([1,2,3,
      4,5,6,
      7,8,0]).
```

% Move definitions (swap blank with valid adjacent tile)

```
move(State, NewState) :-nth0(Blank, State, 0),neighbor(Blank,
SwapWith),swap(State, Blank, SwapWith, NewState).
```

% Valid neighbor positions in 3x3 grid

```
neighbor(Pos, Left) :-Col is Pos mod 3,Col > 0,Left is Pos - 1.
```

```
neighbor(Pos, Right) :-Col is Pos mod 3,Col < 2,Right is Pos + 1.
```

```
neighbor(Pos, Up) :-Pos >= 3,Up is Pos - 3.
```

```
neighbor(Pos, Down) :-Pos =< 5,Down is Pos + 3.
```

% General swap predicate

```
swap(State, I, J, NewState) :-nth0(I, State, A),nth0(J, State,
B),set_elem(State, I, B, Temp),set_elem(Temp, J, A, NewState).
```

% Replace element at index

```
set_elem([_|T], 0, X, [X|T]).
```

```
set_elem([H|T], I, X, [H|R]) :-I > 0,I1 is I - 1,set_elem(T, I1, X, R).
```

% DFS Search

```
dfs(State, _) :-goal(State),  
    write('Goal reached: '), write(State), nl.
```

```
dfs(State, Visited) :-  
    move(State, Next),  
    \+ member(Next, Visited),  
    dfs(Next, [Next|Visited]).
```

% Start predicate

```
solve(Start) :-  
    dfs(Start, [Start]).
```

Output:

```
| ?- solve([1,2,3,4,5,6,0,7,8]).  
Goal reached: [1,2,3,4,5,6,7,8,0]  
  
true ?  
  
(32 ms) yes  
| ?- |
```

Practical 08

Aim: Write a program to solve travelling salesman problem using Prolog.

% distance facts

edge(a, b, 10).

edge(a, c, 15).

edge(a, d, 20).

edge(b, c, 35).

edge(b, d, 25).

edge(c, d, 30).

% symmetric distance

distance(X, Y, D) :- edge(X, Y, D).

distance(X, Y, D) :- edge(Y, X, D).

% cities

cities([a, b, c, d]).

% path cost

path_cost([], 0).

path_cost([X, Y | Rest], Cost) :- distance(X, Y, D), path_cost([Y | Rest], Cost1),

Cost is D + Cost1.

% cycle

tour_cost([Start | Rest], Cost) :-

path_cost([Start | Rest], Cost1),

last(Rest, Last),

distance(Last, Start, D),

Cost is Cost1 + D.

% TSP Solution

tsp(BestPath, MinCost) :-

```
    cities(Cities),  
    permutation(Cities, [Start | Rest]),  
    tour_cost([Start | Rest], Cost),  
    \+ (permutation(Cities, [S2 | R2]),  
        tour_cost([S2 | R2], Cost2),  
        Cost2 < Cost),  
    BestPath = [Start | Rest],  
    MinCost = Cost.
```

Output:

```
compiling C:/AI/practical-8.pl for byte code...  
C:/AI/practical-8.pl compiled, 39 lines read - 4048 bytes written, 17 ms  
| ?- tsp(Path, Cost).  
Cost = 80  
Path = [a,b,d,c] ?
```

Practical 09

Aim: Develop an expert system for medical diagnosis of childhood diseases using prolog.

% Dynamic declaration to store user responses

:- dynamic(yes/1).

:- dynamic(no/1).

disease(measles) :-

symptom(fever),symptom(cough),symptom(runny_nose),symptom(red_eyes),symptom(rash).

disease(chickenpox) :-

symptom(fever),symptom(rash),symptom(blisters),symptom(fatigue).

disease(common_cold) :-symptom(runny_nose),symptom(cough),

symptom(sneezing),\+ symptom(fever).

disease(flu) :-

symptom(fever),symptom(headache),symptom(body_ache),symptom(fatigue),

symptom(cough).

symptom(X) :-yes(X), !.

symptom(X) :- no(X), !, fail.

symptom(X) :-ask(X).

ask(X) :-format('Does the child have ~w? (yes/no): ', [X]),read(Response),

(

(Response == yes ; Response == y)

-> assertz(yes(X))

; assertz(no(X)), fail

).

```
diagnose :- retractall(yes(_)), retractall(no(_)), disease(Disease), format('Possible  
disease: ~w~n', [Disease]), !.
```

```
diagnose :- write('No matching disease found.'), nl.
```

Output:

```
| ?- diagnose.  
Does the child have fever? (yes/no): yes.  
Does the child have cough? (yes/no): yes.  
Does the child have runny_nose? (yes/no): yes.  
Does the child have red_eyes? (yes/no): yes.  
Does the child have rash? (yes/no): yes.  
Possible disease: measles  
  
(16 ms) yes  
| ?-
```

Practical 10

Aim: Write a Prolog program to count even and odd elements from list and count elements up to specific index in list.

```
count_even_odd([],0,0).
```

```
count_even_odd([H|T],Even,Odd):- H mod 2 == 0,
count_even_odd(T,E1,O1), Even is E1+1, Odd is O1.
```

```
count_even_odd([H|T],Even,Odd):- H mod 2 \= 0,
count_even_odd(T,E1,O1), Even is E1, Odd is O1+1.
```

```
| ?- count_even_odd([1,43,2,63,30,24],Even, Odd).
Even = 3
Odd = 3 ? |
```

```
count_upto_index(_,0,0).
```

```
count_upto_index([],_,0).
```

```
count_upto_index([H|T],Index,Count):- Index>0,I1 is Index -
1,count_upto_index(T,I1,C1),Count is C1+1.
```

```
| ?- count_upto_index([10,20,30,40,50], 3, Count).
Count = 3 ? |
```